Digital Systems

Lecture 9

Serial I/O

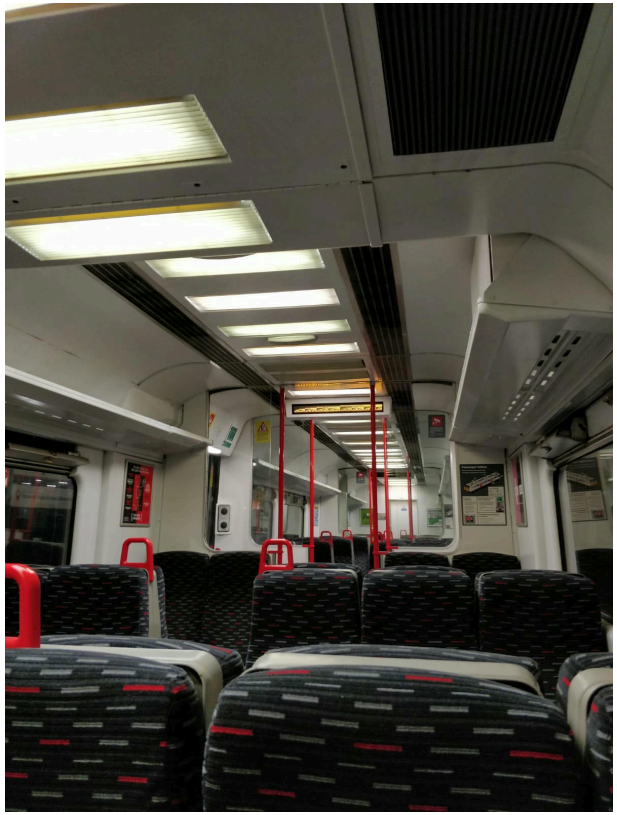
Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

More Chocolate



What is going on here? Chocolate for the best hypothesis.

Digital Systems

Lecture 9

Serial I/O

Mark van der Wilk

Hilary Term 2025

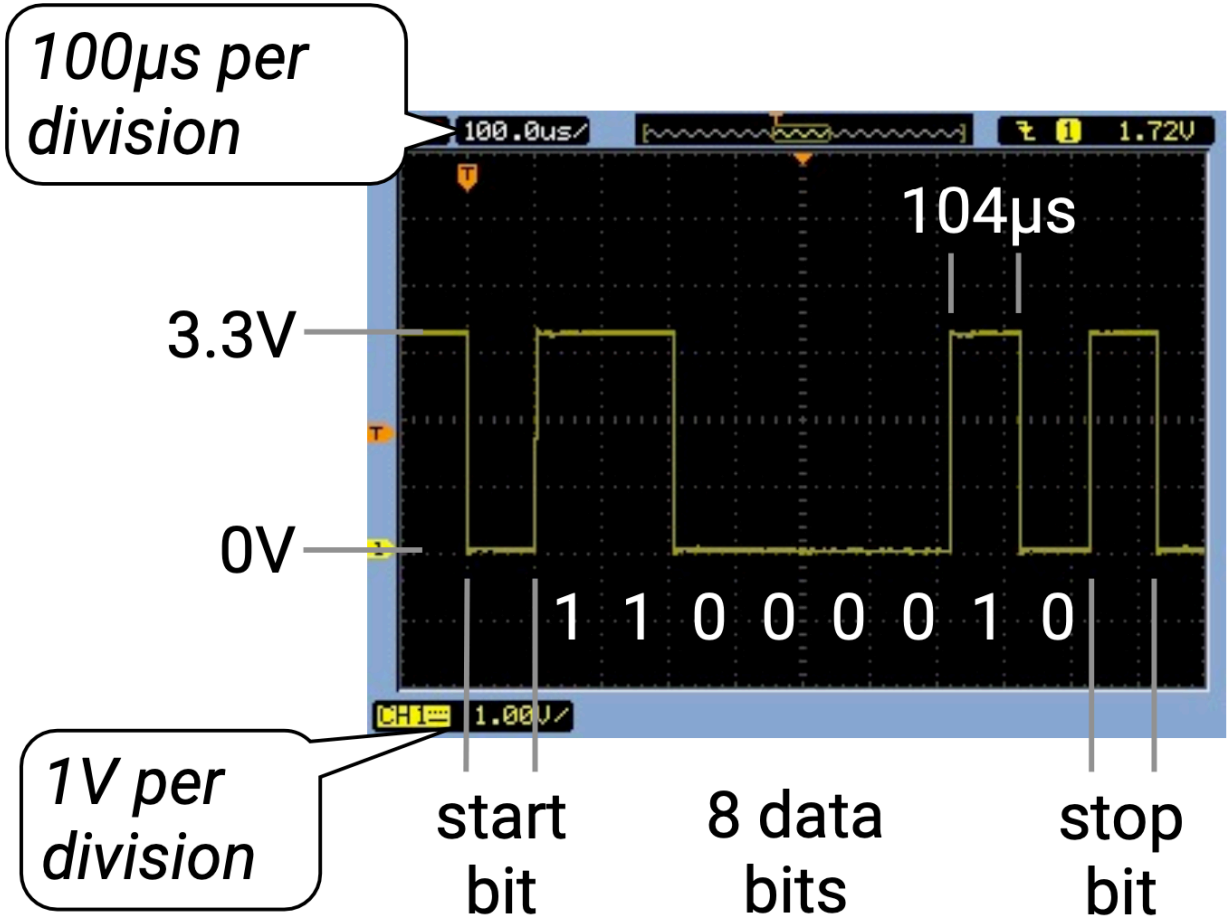
Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

- We have electrical signals at our disposal.
- Design constraint: Let's try to use as few pins as possible.
- Communicate bits sequentially over a single wire.

Serial Communication: Oscilloscope

- Idle: High.
- Start bit: Indicates data transmission.
- Read data at regular time intervals (crucial!).
- Stop bit: Return to high for idle.
- Configured baud rate: 9600 symbols/s (i.e. bits/s).
- Could implement w/ GPIO



UART Interface

On Nordic chip, hardware handles most conversions, with a convenient *memory-mapped* interface.

Universal Asynchronous Receive Transmit interface on the Nordic chip provides:

- Full-duplex operation: Separate receive/transmit wires, for simultaneous receive/transmit.
- Universal: Can operate at different speeds.
- Asynchronous: No shared clock signal (cf. SPI)

A Basic UART Driver: Setup

```
void serial_init(void) {
  UART_ENABLE = 0;
  // Select 9600 Baud, relative to 16 MHz crystal
  UART_BAUDRATE = UART_BAUD_9600;
  UART_CONFIG = UART_CONFIG_8N1;  // 8N1 (8 data,1 stop)
  UART_PSELTXD = USB_TX;          // choose pins
  UART_PSELRXD = USB_RX;
  UART_ENABLE = UART_Enabled;
  UART_RXDRDY = 0; UART_TXDRDY = 0;
  UART_STARTTX = 1; UART_STARTRX = 1;
  txinit = 1;
}
```

A Basic UART Driver: Transmission

```
void serial_putc(char ch) {
    while (! UART_TXDRDY) { /* idle */ } UART_TXDRDY = 0;
    UART_TXD = ch;
}
```

- *Polling* to wait for previous character to finish transmiting.
- `printf()` is wrapper around this.

A Communicating Program

Print out sequence of primes via serial.

```
start_timer();
while (count < 500) {
  if (prime(n)) {
    count++;
    printf("prime(%d) = %d\r\n", count, n);
  }
  n++;
}
stop_timer();
```

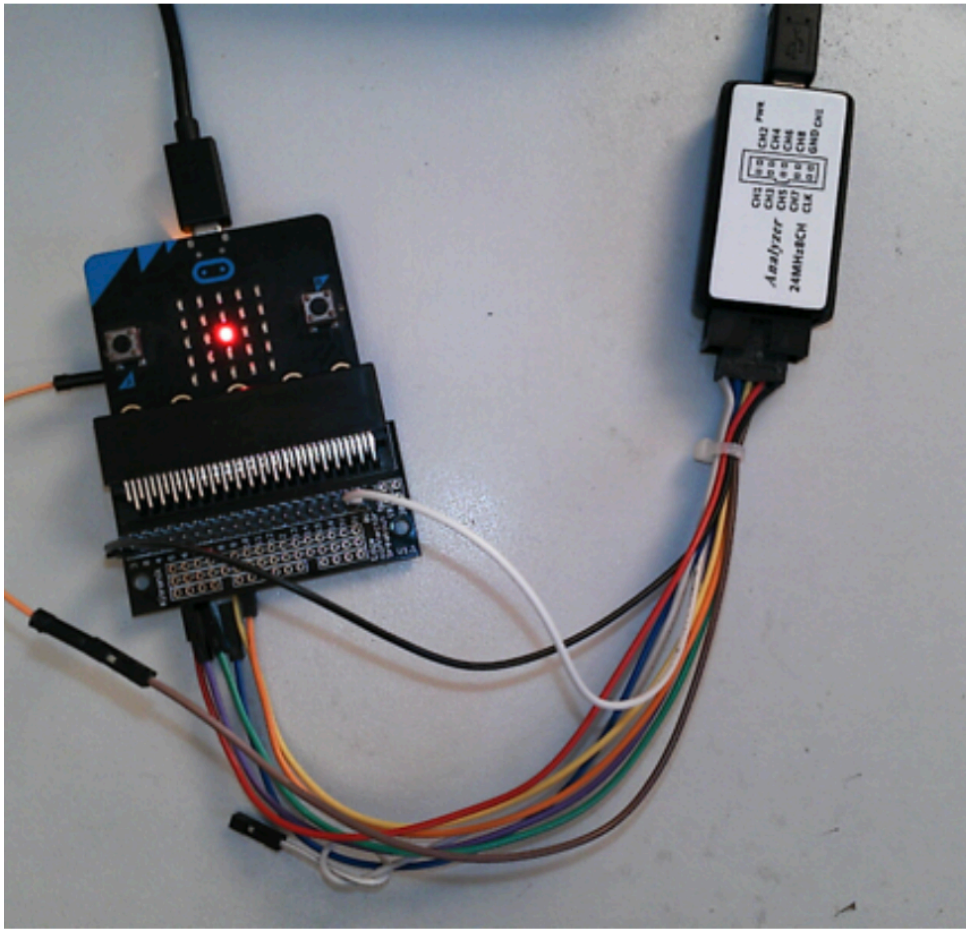
Polling: Spends a lot of time transmitting, rather than calculating!

Timing with Logic Analyser

Can confirm this hunch by looking at signals from pins.

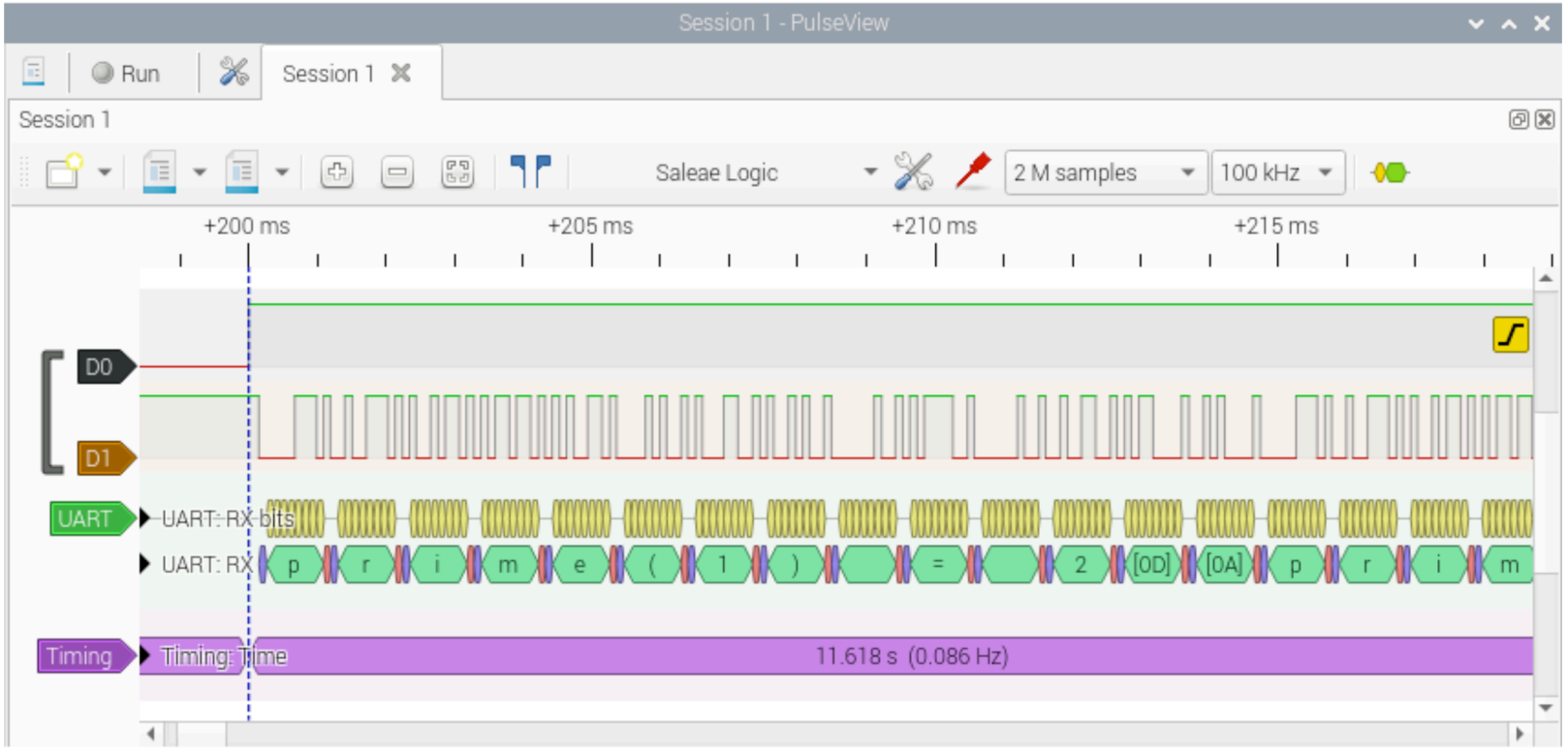
- Like an oscilloscope, but only stores digital state.
- More channels (8).
- Can record for longer time.

Here: Monitors both LED and UART.



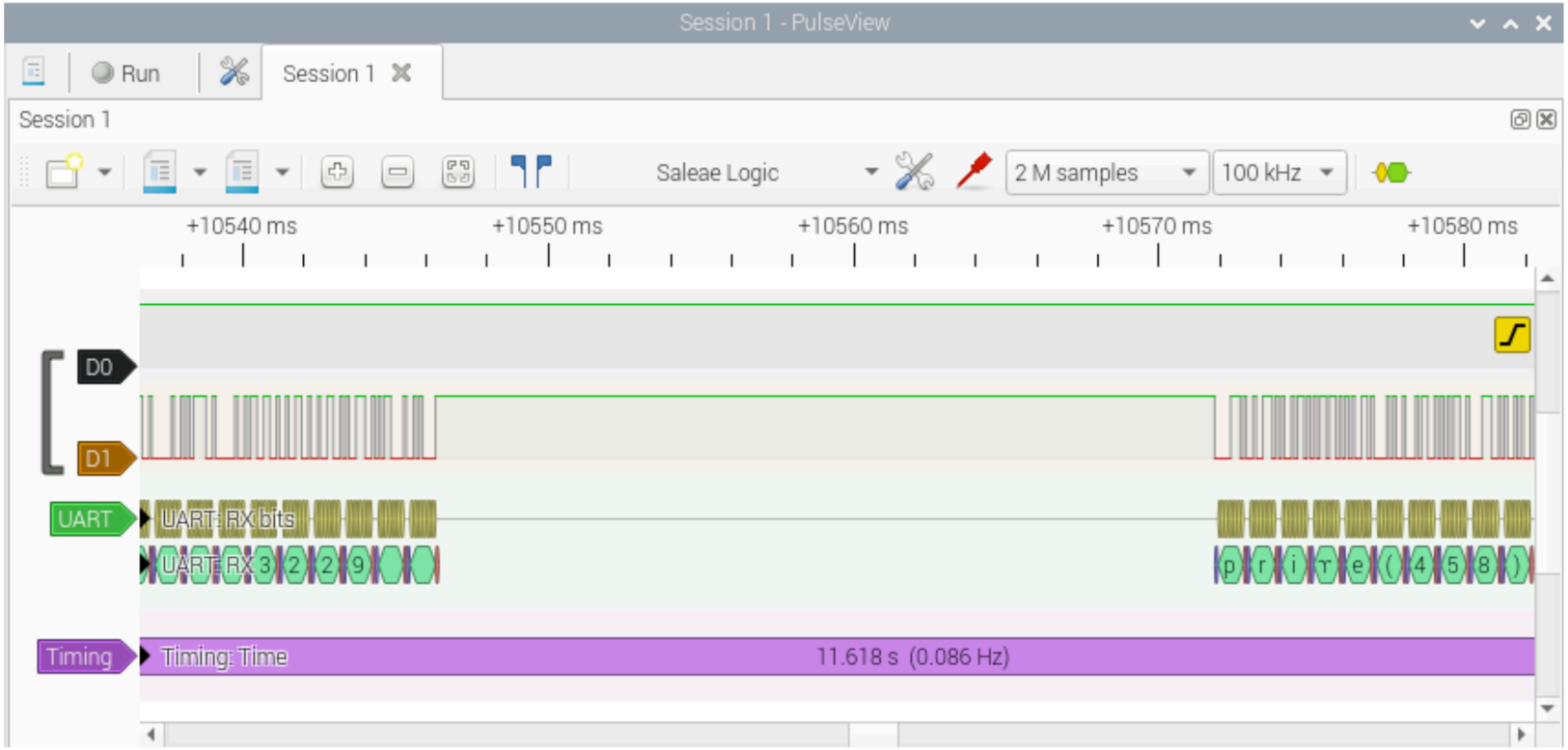
Timing with Logic Analyser

Early on, most time is spent waiting for UART to send.



Timing with Logic Analyser

Later on, longer periods are spent calculating.



? Do we need to wait around while UART is transmitting?

What is the point of having hardware to handle receiving/transmission, if the CPU will just wait around until it's done anyway?

Summary

- Electrical serial protocol.
- Software interface of serial.

Bonus

Remember:

```
unsigned x = GPIO_IN;
if (GET_BIT(x, BUTTON_A) == 0 || GET_BIT(x, BUTTON_B) == 0) {
  // A button is pressed
}
```

Let's look at macros in `hardware.h` in more detail.

- `GPIO_OUT`
- `SET_FIELD(GPIO_PINCNF[BUTTON_A], GPIO_PINCNF_INPUT, GPIO_INPUT_Connect);`
- `GET_BIT`, `SET_BIT`, `CLR_BIT`